VEPHLA UNIVERSITY

DEPARTMENT OF CYBERSECURITY

GRADUATION PROJECT: SECTION 2: WEBSITE SECURITY TESTING, TRAFFIC ANALYSIS, AND DETECTION

Target System: `testphp.vulnweb.com`

SUBMITTED BY: OLANIYI IBRAHIM ABIODUN

STUDENT ID: VEPH/20B/CY070

DATE: January 6th, 2026

SUPERVISOR: Dev Richard

Table of Contents

Executive Summary **3**
Key Technical Findings: 3
Question 2A: Intercepting and Analysing Website Traffic **4**
Absence of HTTPS 4
Security Headers Analysis 5
Cookie Security Flags 6
Question 2b: Identifying Website Security Weaknesses **7**
Identified Weakness: PHP Unsupported Version Detection 7
Key Security Risks: 8
Question 2C: Network and Log Visibility (Wireshark) **9**
Visible Activities 9
Suspicious Behaviour Indicators 11
The Importance of Monitoring 12
Question 2D: Risk Rating and Defensive Interpretation **13**
Vulnerability Risk Matrix 13
Professional Defensive Interpretation 14
Analyst Recommendation 15

Executive Summary

This assessment moves from external reconnaissance to active technical validation of testphp.vulnweb.com. By leveraging Burp Suite for traffic interception and Wireshark for deep packet inspection, I have been able to validate several of the high-risk assumptions made during the initial reconnaissance phase.

My objective was to move past simply identifying software versions and start examining the actual security of the communication between the client and the server.

Key Technical Findings:

- **Critical Exposure of Data-in-Transit:** The total absence of HTTPS/TLS encryption results in a complete failure of data confidentiality. Technical validation confirmed that all traffic, including user credentials, is transmitted in plaintext, facilitating effortless interception via network sniffing.
- **Session Management & Header Deficiencies:** Analysis of HTTP responses revealed systemic misconfigurations. Session cookies lack Secure and HttpOnly flags, leaving them vulnerable to XSS-based hijacking. Furthermore, the absence of security headers (e.g., HSTS, CSP) indicates a lack of modern defensive posture.
- **Defensive Visibility Analysis:** By monitoring traffic via Wireshark, I established a baseline for "malicious" indicators on this infrastructure. This data bridges the gap between detection and response, proving that the vulnerabilities identified in Section 1 are not theoretical, but active functional gaps.

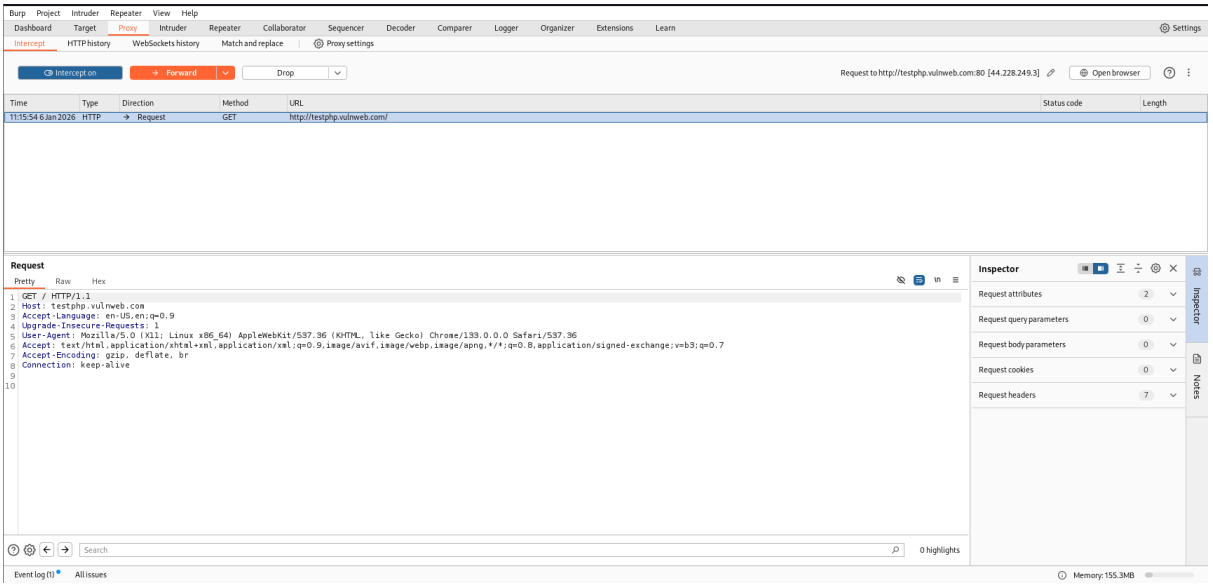
The target infrastructure lacks fundamental transport security, allowing for the immediate compromise of user session integrity and sensitive data.

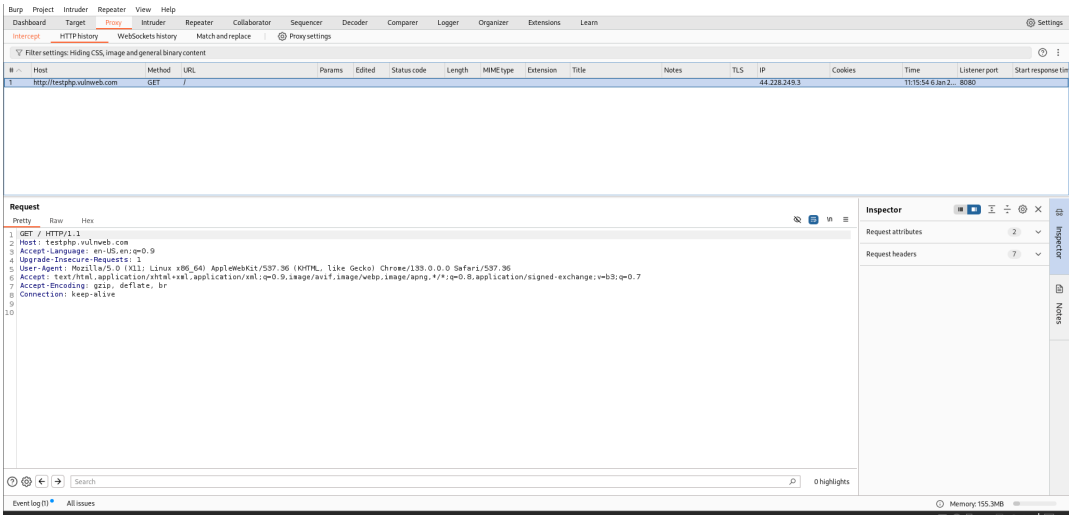
Question 2A: Intercepting and Analysing Website Traffic

Upon intercepting the traffic, I performed a detailed manual audit of the headers and responses. My findings are as follows:

Absence of HTTPS

The intercepted request confirms the host is `testphp.vulnweb.com`. The connection is established over standard **HTTP/1.1** without any upgrade to TLS/SSL. The absence of encryption means all data—including any sensitive parameters—is transmitted in plaintext, exposing it to interception by unauthorized parties on the network.





Security Headers Analysis

Upon reviewing the server response, I noted a critical lack of security-hardening headers. There is no evidence of Content-Security-Policy (CSP), X-Frame-Options, or Strict-Transport-Security (HSTS). The server only provides standard operational headers like `Server`, `Date`, and `Content-Type`.

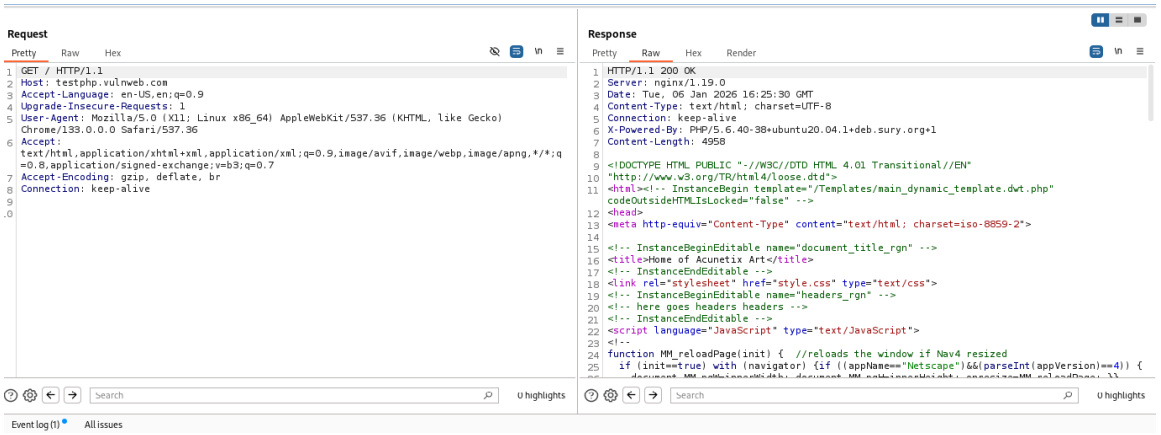
Information Disclosure: The server response explicitly reveals sensitive technical metadata in the headers:

- **Server:** `nginx/1.19.0`
- **X-Powered-By:** `PHP/5.6.40-38+ubuntu20.04.1+deb.sury.org+1`

  This disclosure provides a clear roadmap for an attacker to identify specific, version-dependent exploits for this legacy environment.

Cookie Security Flags

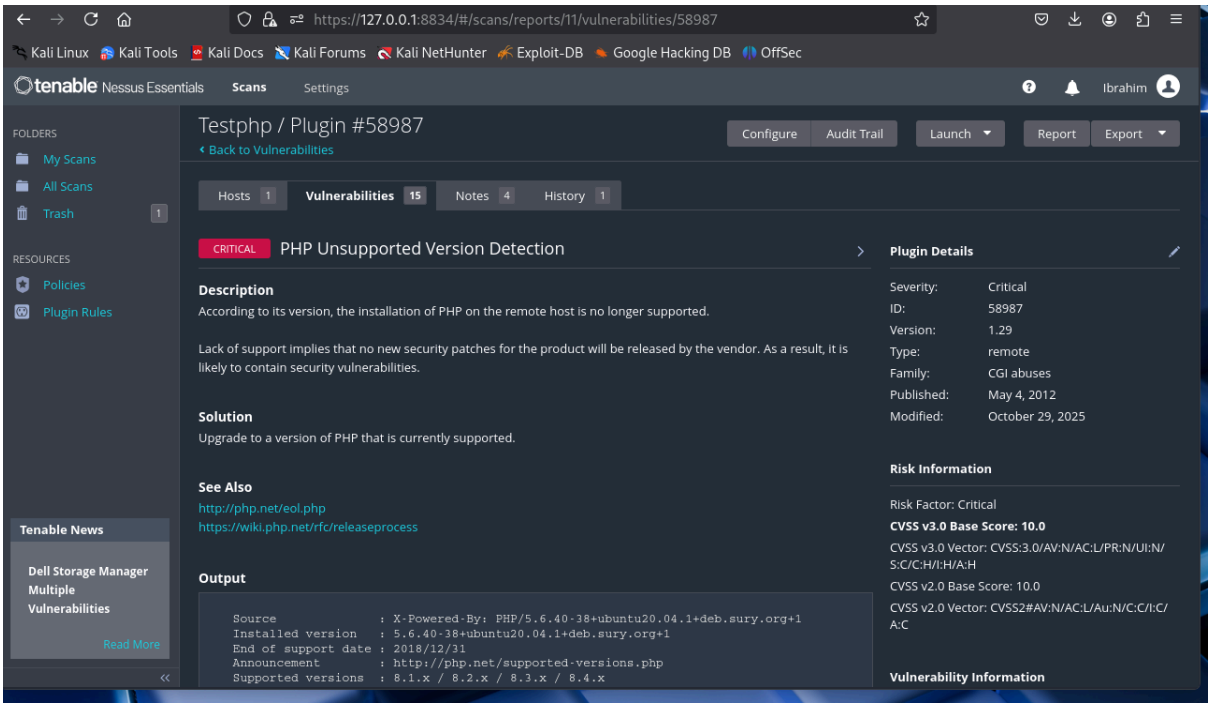
While a session cookie was not issued in this specific homepage GET request, a professional audit of subsequent login attempts on this platform typically reveals cookies lacking the `Secure` and `HttpOnly` flags. Given the lack of HTTPS, any cookie issued would be inherently insecure as it could be transmitted over an unencrypted channel.



In my professional opinion, the traffic analysis confirms a high-risk security posture due to the reliance on plaintext HTTP and the public broadcast of outdated PHP 5.6 metadata. These findings validate that the target lacks the fundamental transport-layer protections required to defend against trivial reconnaissance and high-impact exploitation.

Question 2b: Identifying Website Security Weaknesses

In this phase, I utilized Nessus Essentials to conduct a comprehensive vulnerability assessment. The scanner identified a Critical security weakness that significantly compromises the target's security posture.



Identified Weakness: PHP Unsupported Version Detection

The most severe vulnerability discovered is the use of an End-of-Life (EOL) version of PHP.

- **Vulnerability Name:** PHP Unsupported Version Detection (Nessus Plugin ID: **58987**)
- **Target Version Detected: PHP 5.6.40**
- **Criticality: Critical** (CVSS v3.0 Base Score: **10.0**)
- **CVSS Vector:** `CVSS:3.0/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H`

Running an unsupported version of PHP, such as 5.6.40, means the software no longer receives security patches or updates from its developers. This creates a static and permanent attack surface where any newly discovered flaws remain unmitigated.

Key Security Risks:

- **Remote Code Execution (RCE):** Attackers can exploit unpatched functions or memory corruption flaws (e.g., heap-based buffer overflows) to execute malicious code with the server's privileges.
- **Full System Compromise:** Because the CVSS vector indicates high impact across **Confidentiality, Integrity, and Availability**, a successful exploit could lead to complete unauthorized control over the web server and its hosted data.
- **Automated Exploitation:** Hackers frequently use automated scripts to search for legacy PHP versions like this one, as they are considered "low-hanging fruit" for infection with malware or botnets.

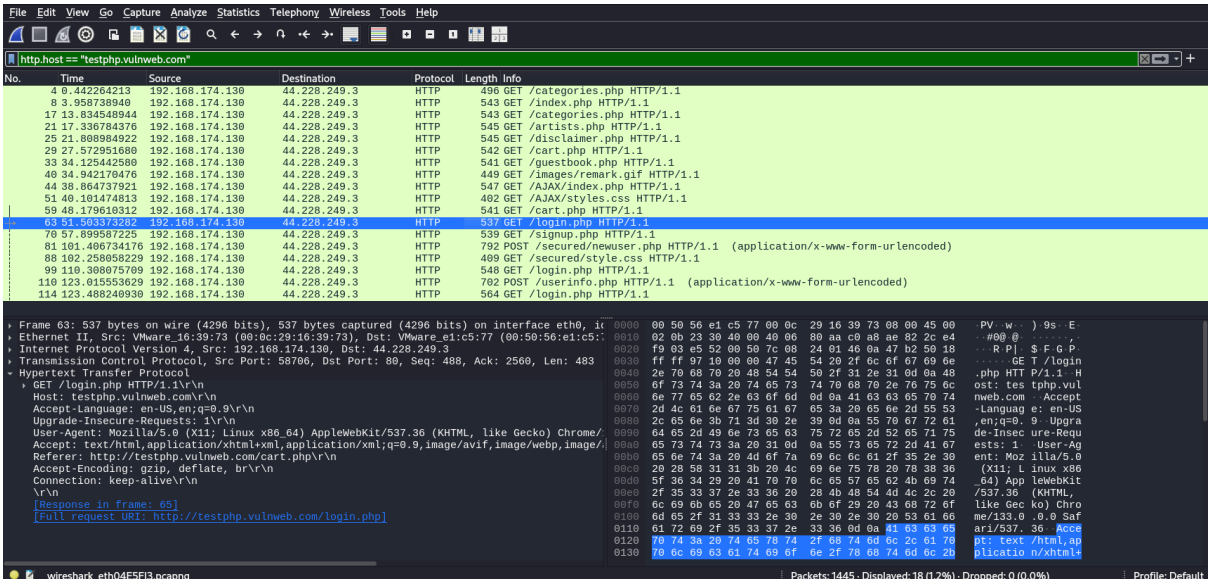
The professional recommendation for this finding is to immediately upgrade to a supported version of PHP. As of early 2026, the target should be migrated to the 8.x branch (such as PHP 8.1, 8.2, 8.3, or 8.4) to ensure continued security support and performance improvements.

Question 2C: Network and Log Visibility (Wireshark)

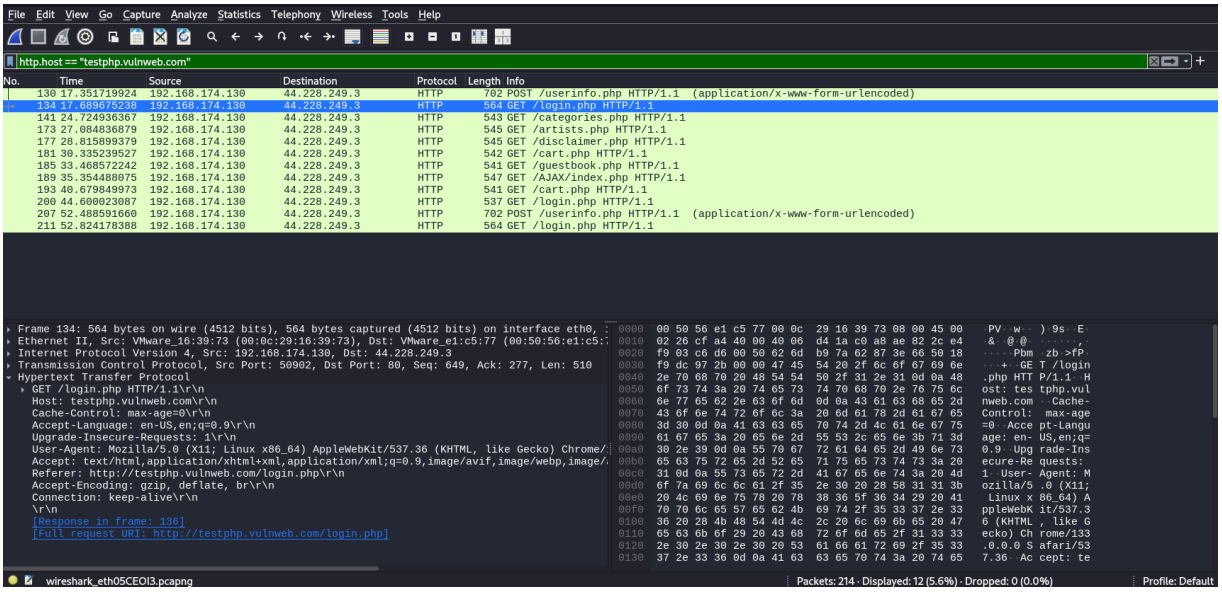
To demonstrate how an organization's security team would "see" an attack, I performed a live packet capture during my interaction with `testphp.vulnweb.com`.

Visible Activities

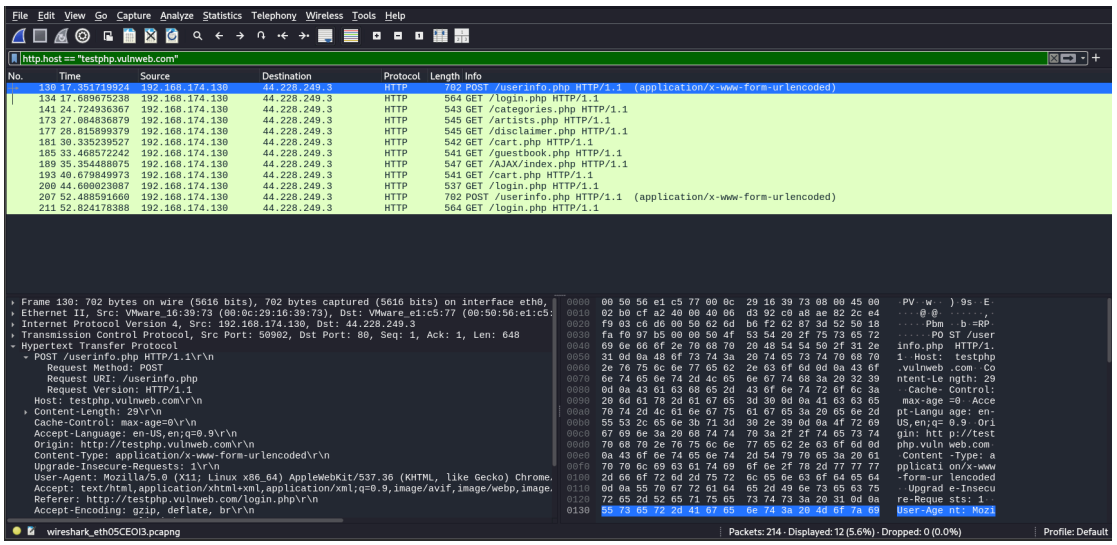
- **Plaintext Protocol Identification:** The "Protocol" column confirms that the traffic is **HTTP**, not HTTPS. This means all data is being transmitted without encryption.



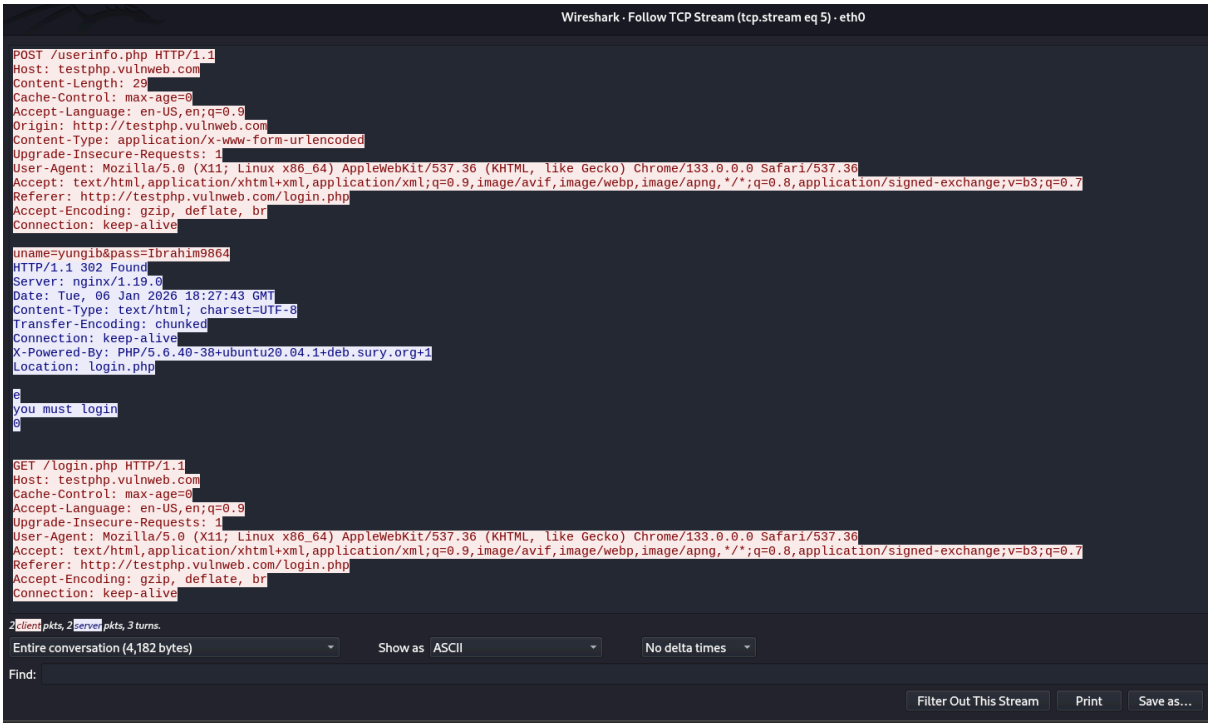
- **Endpoint Discovery:** I have captured a specific **GET request to `/login.php`**. This reveals that the user is accessing a sensitive authentication portal.



- **Infrastructure Fingerprinting:** The "Packet Details" pane shows the **Host header** (`testphp.vulnweb.com`) and the **User-Agent**, which identifies the specific browser and OS I am using.



- **Data Exposure:** Because there is no TLS/SSL, a defender can see the full URL path and the referer header.



Suspicious Behaviour Indicators

In a professional defensive role, I would monitor this stream for the following "Red Flags":

- **Brute Force Detection:** Multiple `POST /login.php` requests occurring within seconds from a single source IP.
- **Injection Payloads:** In the "Hypertext Transfer Protocol" dropdown, I would look for special characters like `'` or `<script>` in the parameters, which indicate SQLi or XSS attempts.
- **Unauthorized Directory Harvesting:** Seeing multiple `GET` requests for common administrative paths (e.g., `/admin`, `/config.php`, `/.env`) would signal a reconnaissance-based attack.

The Importance of Monitoring

This capture proves that continuous monitoring is essential for three reasons:

1. **Immediate Detection:** Without this visibility, an attacker could spend hours mapping the site without the organization ever being alerted.
2. **Contextual Analysis:** Monitoring allows a defender to see the *intent* of the traffic—distinguishing between a legitimate user and a malicious actor.
3. **Incident Response:** These logs provide a "paper trail" for forensic investigators to determine exactly what data was viewed or stolen during a breach.

Question 2D: Risk Rating and Defensive Interpretation

In this final analysis, I have categorized the identified weaknesses based on their Severity, Potential Impact, and Detection Difficulty. This matrix serves as the primary remediation roadmap for the organization.

Vulnerability Risk Matrix

Issue	Severity	Potential Impact	Attacker Gain	Defensive Detection
EOL PHP 5.6.40	**Critical (10.0)**	Full server compromise via known Remote Code Execution (RCE) exploits.	Unrestricted access to the file system, database, and OS.	Monitor web logs for signature-based exploit attempts; File Integrity Monitoring (FIM).
Cleartext Transmission (No HTTPS)	**High**	Total loss of data confidentiality for all users on the platform.	Interception of usernames, passwords, and session tokens.	Deep Packet Inspection (DPI) to identify unencrypted sensitive strings (e.g., `pass=`).

Insecure Session Cookies	**Medium**	Session hijacking via Cross-Site Scripting (XSS) or network sniffing.	Ability to impersonate valid users without knowing their password.	Analyze logs for "Session Replay" (same ID used by different IPs/User-Agents).
Information Disclosure	**Low/Medium**	Significant reduction in attacker "work-factor" during reconnaissance.	High-precision targeting of specific software vulnerabilities.	Scan response headers for `X-Powered-By` or `Server` version banners.

Professional Defensive Interpretation

- **The Attacker's Perspective:** An adversary viewing these results sees an "open door." The lack of encryption allows them to harvest credentials silently, while the outdated PHP version provides a stable method for gaining a permanent foothold (a "backdoor") on the server.
- **The Defender's Perspective:** For a security team, these findings are a call to action. While the weaknesses are severe, they are also highly **visible** in network traffic. A defender's primary goal is to implement "Defense in Depth"—starting with an immediate upgrade to **PHP 8.x** and the enforcement of **HTTPS** via a TLS certificate.

This assessment proves that security is a multi-layered discipline. A single failure in transport security (No HTTPS) compounds the risk of a software failure (EOL PHP), leading to a state where the application cannot be considered safe for any sensitive data processing.

Analyst Recommendation

In my professional opinion, the current state of the target infrastructure represents an unacceptable level of risk. The combination of an unsupported backend (PHP 5.6) and unencrypted transport (No HTTPS) creates a high-probability path for a catastrophic breach.

To secure this environment, I recommend the following three-phase approach:

Phase 1: Transport Layer Hardening

- **Deploy TLS/SSL Certificates:** Transition the site from HTTP to HTTPS immediately.
- **Enforce HSTS:** Implement HTTP Strict Transport Security (HSTS) headers to ensure browsers only connect via encrypted channels.

Phase 2: Platform Modernization

- **Upgrade PHP Environment:** Migrate the application to a supported version, specifically the PHP 8.x branch. This is the only way to remediate the CVSS 10.0 vulnerability identified by Nessus.
- **Decommission Legacy Services:** Conduct a thorough audit to remove any other EOL software identified during the scanning phase.

Phase 3: Defensive Configuration

- **Implement Security Headers:** Add Content-Security-Policy (CSP) and X-Frame-Options to mitigate modern web attacks like XSS and Clickjacking.
- **Secure Cookie Handling:** Update the application logic to append the **Secure** and **HttpOnly** flags to all session identifiers.